Predicting Daily Station-Level Citi Bike Demand: An End-to-End Machine Learning Pipeline

Team 10
Helena Li (yl6029) Wentao Zhong (wz2753) Kevin Ma (km4189) Ketaki Dabade (kvd2112)
Project 4 – STAT GR5243

Abstract. We present an end-to-end machine learning pipeline for predicting daily trip demand at the station level across the New York City Citi Bike network. Starting from raw web-scraped trip records and weather API data, we construct a station-day analytic panel of 300 stations spanning January and February 2026, apply K-Means clustering ($k = 4$) to encode station behavioral profiles as supervised features, and engineer a rich feature set comprising lag demand, cyclical calendar encodings, nonlinear weather indicators, and text-derived neighborhood context. Four supervised models are trained and compared under a strict time-based holdout. The Tuned Random Forest achieves the best test performance (MAE ≈ 19.75, RMSE ≈ 28.13, $R^2 \approx 0.289$), with prior-day demand and temperature as dominant predictors. An interactive Shiny dashboard is accessible at helenaliyz.shinyapps.io/citibike-dashboard. All code and data are available at github.com/Helenaliyz/5243Project4.

1. INTRODUCTION

Citi Bike is New York City's largest shared-mobility system, logging millions of rides each year across hundreds of docking stations spanning Manhattan, Brooklyn, Queens, and the Bronx. Behind every ride is a daily logistics operation: fleet managers must physically redistribute bicycles among stations to prevent empty docks at high-demand hubs and full docks at low-demand ones. When forecasts are accurate, rebalancing trucks can be pre-positioned efficiently; when they fail, the consequences are visible immediately to riders and costly to operators. Accurate station-level demand prediction is therefore not an academic exercise but an operational necessity.

The forecasting problem is harder than it first appears. Demand at any given station on any given day is shaped by at least three interacting forces: *temporal patterns* (commuting rhythms, weekend leisure cycles, week-to-week persistence), *exogenous shocks* (temperature swings, rain, wind), and *station identity* (a midtown commuter hub in Hell's Kitchen and a waterfront leisure dock at Pier 61 follow fundamentally different demand regimes even on the same day). A model that ignores station identity will systematically mispredict both types. The distribution of station-day demand is also highly right-skewed, with a heavy tail of very busy stations that standard linear models handle poorly without transformation.

This project addresses these challenges through a complete, reproducible data science pipeline. We acquire and merge three heterogeneous data sources, apply meticulous cleaning and aggregation, use unsupervised clustering to discover and encode station identity, engineer a feature set designed to capture all three demand drivers, and build and compare four supervised regression models with principled validation. The result is a well-performing predictive model alongside a transparent, interpretable account of what drives Citi Bike ridership, with findings that are directly actionable for operations planning.

2. DATA ACQUISITION AND PREPARATION

2.1 Data Sources

This study combines three data sources of deliberately different types and acquisition routes, reflecting the multi-source, mixed-structure nature of real-world data science problems. The primary source is the Citi Bike trip record archive, a public repository of monthly CSV files hosted on Amazon S3. Each file records one row per individual ride and contains a ride identifier, precise start and end timestamps, origin and destination station identifiers and canonical display names, station GPS coordinates, rider membership classification (`member` or `casual`), and bicycle type (`classic_bike` or `electric_bike`). We focus on January and February 2026, two months that together span nine complete weekday–weekend cycles and bracket a range of winter meteorological conditions.

The second source is the Open-Meteo Historical Weather API (`https://open-meteo.com`), which provides retrospective daily meteorological summaries at arbitrary geographic coordinates. We queried the API at the centroid of the Citi Bike service area (40.73° N, −73.99° W) for `temperature_2m_mean`, `precipitation_sum`, and `windspeed_10m_max` over the study period, receiving a clean JSON response that required no pre-processing before integration.

The third source is unstructured and implicit: station

metadata. Because Citi Bike does not publish a standalone station reference table for the 2026 data vintage, station identifiers, display names, and coordinates were reverse-engineered from the trip records themselves. Neighborhood proximity indicators were then derived through text parsing of station names, as described below.

2.2 Data Acquisition

Rather than downloading files manually, we built a fully automated Python acquisition pipeline. The Citi Bike archive index at https://s3.amazonaws.com/tripdata/index.html is an HTML page listing all available downloads. We fetched this page with the `requests` library and parsed it with `BeautifulSoup`, extracting every hyperlink whose text matched the target month pattern. The two target URLs were identified programmatically and streamed to disk in 1 MB chunks via `requests.get(..., stream=True)`, avoiding the memory overhead of loading each compressed archive into RAM at once.

The raw CSVs were read directly from the zip containers using Python's built-in `zipfile` module, and only the columns required for downstream analysis were retained via `usecols` filtering. Processing proceeded one month at a time, with the raw `DataFrame` discarded immediately after aggregation to keep peak memory consumption low. This design made the pipeline runnable on standard consumer hardware without cloud storage or distributed computing infrastructure.

Weather data were retrieved with a single parameterized GET request to the Open-Meteo historical API endpoint. The JSON response was parsed directly into a `pandas DataFrame` and required no further transformation before merging on the `date` key.

2.3 Data Cleaning

The raw trip CSVs presented several data quality challenges before aggregation could proceed. Timestamp fields (`started_at`, `ended_at`) were cast to `pandas Timestamp` objects; records with unparseable or out-of-range values were discarded. Rows with null or syntactically malformed `start_station_id` values were removed, addressing a known issue in Citi Bike exports where docking failures produce missing station identifiers. Station names were stripped of whitespace, lowercased for matching, and standardized to a single canonical form per station identifier (the modal value across all trips at that station).

Station coordinates were validated for geometric consistency: any station whose coordinate values exhibited variance above a small tolerance threshold had its coordinates replaced with the per-station median. Duplicate trip records were dropped.

A notable step was the derivation of neighborhood context features from unstructured station names. We applied keyword matching to the standardized name field to create four binary indicators: `has_park`, `has_pier`, `has_plaza`, and `has_square`. Though simple, this text-based approach encodes meaningful environmental signal: stations near parks and waterfronts exhibit systematically higher casual-rider proportions and stronger weekend demand lift, as confirmed by the clustering results in Section 4.

2.4 Aggregation and Panel Construction

Following cleaning, individual trip records were collapsed to a station-day panel by grouping on the composite key (`station_id`, `date`). For each cell we computed `daily_trip_count` as the total number of departures, `casual_share` and `member_share` as the proportions of rides by rider type, and `classic_bike_share` and `electric_bike_share` as the proportions by bicycle type. Each share pair sums to unity by construction. To ensure sufficient data density, we retained only the 300 stations with the highest cumulative trip count. The daily weather table was then left-joined on `date`, and station-level context was left-joined on `station_id`. The resulting analytic table contains $300 \times 59 = 17{,}700$ station-day rows and serves as the sole input to all subsequent analysis.

3. EXPLORATORY DATA ANALYSIS

Exploratory analysis of the cleaned station-day panel pursued two goals: generating descriptive insights about system-wide and station-level demand patterns, and identifying structure in the data that would directly inform modeling choices.

3.1 System-Wide Demand Trend

Figure 1 plots total daily system trips, colored by weekday versus weekend. Total demand ranges from near zero in early January to a peak of roughly 48,000 trips around January 20, then oscillates between approximately 5,000 and 35,000 trips/day through February. The clear weekday–weekend separation evident throughout the series confirms that day-of-week is a first-order predictor of demand. The non-stationarity of the trend also rules out naive global means as a forecasting baseline, reinforcing the decision to use per-station lag features that implicitly track each station's local demand level over time.

3.2 Demand Distribution and Target Transformation

The histogram and boxplot in Figure 2 reveal a single-mode, highly right-skewed distribution of station-day trip counts. The modal bin contains over 4,000 observations near zero, yet the distribution has a long tail extending beyond 400 trips. The boxplot confirms a median of approximately 40 trips/day and a large outlier cloud above the upper fence. This skew has

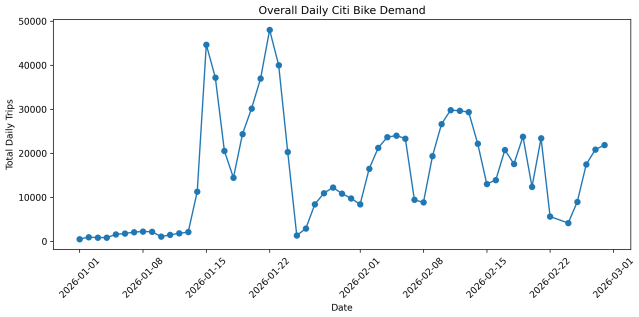

Figure 1: System-wide total daily departures, Jan–Feb 2026, colored by weekday (blue) vs. weekend (red). The weekday premium and within-week periodicity are consistent throughout the observation window.

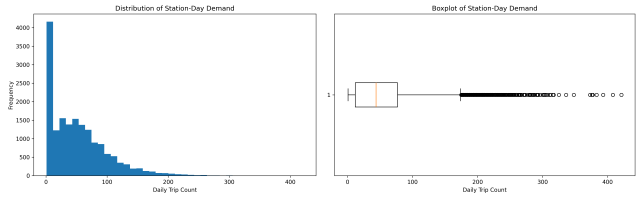

Figure 2: Left: histogram of station-day trip counts (right-skewed, long tail). Right: boxplot confirming median ≈ 40 trips/day and heavy outlier population. Both motivate the log-target transformation.

direct modeling implications: it violates the homoskedasticity assumption of ordinary least squares and causes tree-based models to allocate disproportionate splits to the high-volume tail. Applying the $\log(1+y)$ transformation substantially compresses this skew and distributes residual variance more evenly across the demand range, justifying the log-scale modeling approach in Section 5.

3.3 Weekday Versus Weekend Demand

Figure 3 compares demand distributions across day types. Per-station median demand is approximately 50 trips on weekdays versus 30 on weekends, and average total weekday trips ($\approx$17,500/day) exceed weekend trips ($\approx$10,800/day) by roughly 60%, consistent with a commuter-dominated ridership base. This difference motivates both the `is_weekend` binary feature and the cyclical day-of-week encodings that allow the model to represent smooth within-week demand rhythms.

3.4 Station-Level Heterogeneity

Figure 4 shows that the top 10 stations cluster into two distinct geographic-functional groups: midtown and Chelsea commuter corridors (*W 21 St & 6 Ave*, *W 31 St & 7 Ave*) and waterfront or recreational nodes (*Pier 61 at Chelsea Piers*). This geographic and behavioral heterogeneity is the key motivation for the station clustering analysis in Section 4: a model that treats all stations identically will systematically mispredict both types.

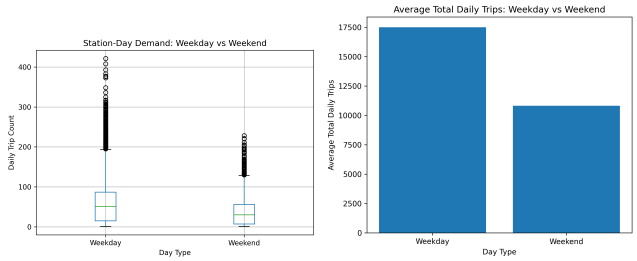

Figure 3: Left: per-station-day trip count distributions by day type. Right: average system-wide daily trips on weekdays vs. weekends. The $\sim$60% weekday premium reflects the commuter-dominated ridership base.

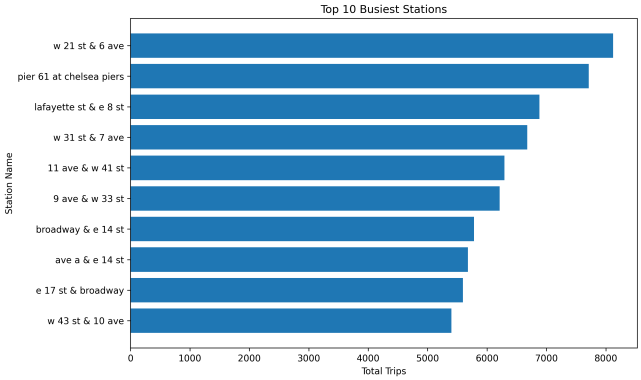

Figure 4: Top 10 stations by total departures, Jan–Feb 2026. High-volume stations span both commuter corridors and waterfront leisure hubs, illustrating the behavioral heterogeneity that motivates clustering.

3.5 Weather Effects on Demand

Figure 5 displays scatter plots of total daily system trips against mean temperature (left) and precipitation (right) with a fitted linear trend. For temperature, the highest-demand days are concentrated at the warmer end of the range, while cold days are uniformly low. For precipitation, virtually every day above 10 mm is a low-demand day, while peak-demand days cluster near zero. Critically, both relationships appear nonlinear, with demand dropping sharply beyond a threshold rather than declining proportionally. This motivates the inclusion of binary precipitation and wind flags in addition to raw continuous weather variables.

3.6 Correlation Structure

Figure 6 presents the Pearson correlation matrix across demand, share, and weather variables. `daily_trip_count` shows the expected weak positive association with `member_share` and weak negative associations with `precipitation_sum` and `max_wind_speed`. More instructive are two artefactual patterns: the -1 correlation between complementary share pairs (`casual_share`/`member_share` and `classic_bike_share`/`electric_bike_share`) is mechanical, as they sum to unity, signaling collinearity rather than predictive signal. The near-zero correlation

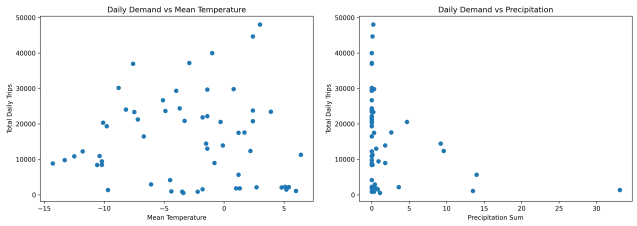

Figure 5: Total daily trips against mean temperature (left) and precipitation (right) with OLS trend lines. Nonlinear threshold effects motivate binary flag features alongside raw continuous variables.

of raw weather variables with trip counts at the station-day level underscores why individual station features, particularly lag demand, will be more predictive than system-level weather alone.

Together, these EDA findings establish three empirical priors: day-of-week and recent demand history are first-order predictors; weather matters through nonlinear threshold effects; and station identity introduces systematic heterogeneity that cannot be addressed through global features alone.

4. UNSUPERVISED LEARNING: STATION CLUSTERING

4.1 Motivation and Method

EDA revealed that the 300 stations differ substantially in volume, temporal rhythm, and rider composition, yet these differences resist simple characterization by latitude and longitude alone. Two stations a few blocks apart can serve entirely different populations, and their demand responds differently to weather, weekends, and season. To encode this station identity in a form the supervised model can use, we applied K-Means clustering to a station-level profile of long-run behavioral characteristics: average, median, and maximum daily trips; coefficient of variation; mean casual and member rider shares; mean electric-bike share; weekend-to-weekday demand ratio; GPS coordinates; and the four neighborhood proximity flags. All 14 features were standardized with `StandardScaler` before clustering.

4.2 Selecting k

We evaluated $k = 2, \ldots, 8$ using both the elbow plot (within-cluster inertia) and average silhouette score. The elbow plot shows a clear inflection at $k = 4$, where the marginal reduction in inertia diminishes sharply. The silhouette curve peaks at $k = 4$ with a score of approximately 0.38, confirming that $k = 4$ yields well-separated, internally cohesive clusters. We therefore fit a final $k = 4$ model with `n_init=10` random restarts. PCA was applied to visualize cluster structure in two dimensions, with two principal components explaining approximately 52% of total variance.

Table 1: Station cluster profiles, from high-volume commuter hubs (C0) to park/leisure docks (C3).

Cluster	#	Avg Trips	Casual	E-Bike	Wknd/Wkdy	Park%
C0: High-Traffic	50	82.71	0.073	0.711	0.608	0.00
C1: Mod. Commuter	109	49.22	0.071	0.731	0.511	0.00
C2: Mod. Leisure	132	45.05	0.073	0.702	0.761	0.02
C3: Park/Leisure	9	48.23	0.192	0.754	0.559	1.00

4.3 Cluster Interpretation

Table 1 summarizes key statistics per cluster. **Cluster 0 (High-Traffic Commuter, 50 stations)** has the highest average daily demand (82.7 trips), a high member share (71.1%), and a below-average weekend/weekday ratio (0.61), identifying it with dense commuter corridors in midtown and lower Manhattan. **Cluster 1 (Moderate Commuter, 109 stations)** shares the low weekend ratio and high member share of Cluster 0 but at lower volume (49.2 trips/day), representing secondary commuter hubs in surrounding neighborhoods. **Cluster 2 (Moderate Leisure, 132 stations)** has a higher weekend/weekday ratio (0.76) and encompasses mixed-use residential areas where weekend cycling is more prominent. **Cluster 3 (Park/Leisure, 9 stations)** is small but behaviorally distinct: the highest casual share (19.2%), 100% park proximity, and moderate demand (48.2 trips/day), identifying waterfront and recreational docks where leisure riding dominates.

The cluster label ($c \in \{0, 1, 2, 3\}$) was one-hot encoded and appended as a categorical feature to the supervised model's input matrix. This creates a principled bridge between the unsupervised and supervised stages of the pipeline: rather than relying solely on GPS coordinates to proxy station character, the model can exploit a compact, data-driven summary of each station's long-run behavioral identity. Critically, the cluster features were computed from the full two-month station profile, not from the same-day data, so they introduce no leakage into the target variable.

5. FEATURE ENGINEERING AND PREPROCESSING

5.1 Target Transformation

As motivated by the EDA distributional analysis, we modeled $\tilde{y} = \log(1 + \texttt{daily\_trip\_count})$ rather than the raw count. This transformation compresses the right tail, reduces heteroskedasticity, and ensures that models fitted under squared loss are not dominated by the handful of extreme high-volume station-days. All evaluation metrics were computed on the original scale by applying the inverse $\hat{y} = \max(0,\ e^{\hat{\tilde{y}}} - 1)$, which preserves interpretability as absolute trip counts.

5.2 Temporal and Calendar Features

Day-of-week was encoded in two complementary forms. The binary `is_weekend` indicator captures the

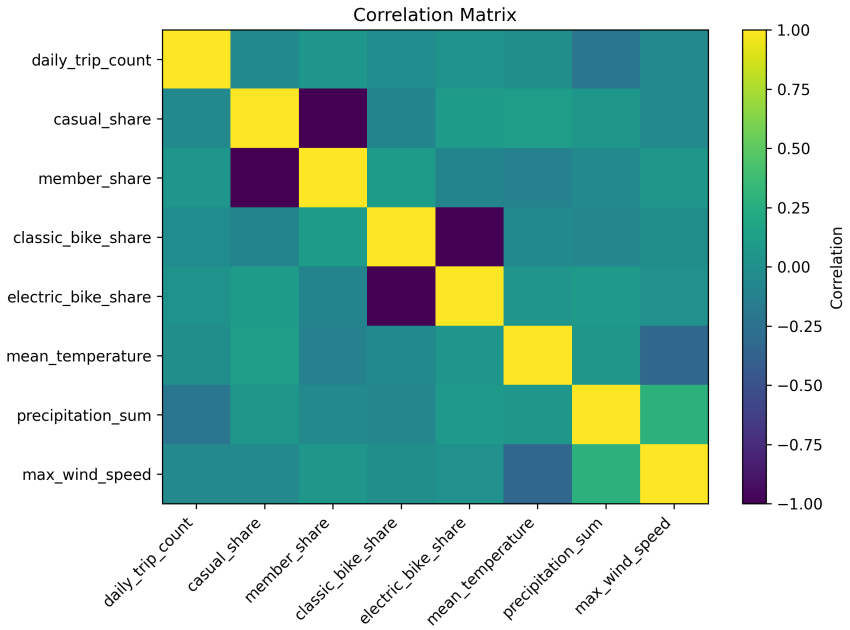

Figure 6: Pearson correlation matrix of demand, rider/bike-type shares, and weather variables. Mechanical -1 correlations between complementary share pairs signal collinearity to be managed during feature selection.

discontinuous commuter/leisure regime shift identified in EDA. Cyclical sine–cosine encodings, `dow_sin` $= \sin(2\pi d/7)$ and `dow_cos` $= \cos(2\pi d/7)$, preserve the circular topology of the week: Sunday and Monday are correctly represented as adjacent rather than maximally distant, as they would be under a raw integer encoding. This design choice is especially important for tree-based models, which partition on axis-aligned thresholds and would otherwise treat the week boundary as a hard discontinuity.

5.3 Weather Features

Raw continuous weather variables (`mean_temperature`, `precipitation_sum`, and `max_wind_speed`) were retained as direct predictors. To capture the nonlinear threshold effects identified in EDA, three binary indicators were added: `has_precipitation` ($= 1$ if precipitation > 0), `heavy_precipitation` ($= 1$ if precipitation exceeds the 90th percentile of the training distribution), and `high_wind` ($= 1$ if wind speed exceeds the 90th percentile). These flags allow tree-based models to cleanly partition precipitation regimes that are visible in the scatter plots but poorly captured by linear functions of the continuous variable alone.

5.4 Lag Demand Features

Two per-station lag features were constructed: `lag_1_trip_count` (trips at the same station on the prior calendar day) and `lag_7_trip_count` (trips on the same weekday of the previous week). Lag-1 captures short-term autocorrelation and absorbs station-specific unobserved factors that persist day-to-day; lag-7 captures weekly seasonality, the strong pattern whereby every Tuesday at a given station resembles the previous Tuesday more than the adjacent Wednesday. Both features were computed separately within each `station_id` group to prevent cross-station contamination. Records missing a valid lag value were dropped prior to modeling.

5.5 Spatial and Context Features

Station coordinates (`start_lat`, `start_lng`) provide a continuous spatial signal, allowing the model to learn smooth geographic demand gradients. The four neighborhood binary flags (`has_park`, `has_pier`, `has_plaza`, `has_square`) add discrete environmental context not fully captured by coordinates alone. The station cluster label (one-hot encoded) supplements both, providing a data-driven summary of station identity at a resolution coarser than individual coordinates but finer than a simple urban/suburban distinction.

5.6 Leakage Prevention and Preprocessing Pipeline

Same-day share variables (`casual_share`, `member_share`, `classic_bike_share`, `electric_bike_share`) were excluded from the supervised feature matrix. Although these are strong correlates of trip count, they are derived from the same set of rides as the target and are therefore unavailable at prediction time. Their station-level long-run averages were legitimately used in the clustering stage, since they summarize historical station behavior over the full window.

All numeric features were standardized using `StandardScaler`, and the cluster label was one-hot encoded with `OneHotEncoder(handle_unknown='ignore')`. Both

Table 2: Test-set performance of all four models, sorted by RMSE.

Model	MAE	RMSE	R^2
Tuned Random Forest	**19.75**	**28.13**	**0.289**
Random Forest	21.23	29.70	0.207
Gradient Boosting	24.70	32.71	0.038
Ridge Regression	25.85	35.78	−0.151

transformers were wrapped in a `ColumnTransformer` embedded within an `sklearn Pipeline`, guaranteeing that the scaler is fit exclusively on training data and applied to test data without information leakage.

6. SUPERVISED MODELING

6.1 Problem Framing and Validation Strategy

The task is a regression on $\tilde{y} = \log(1 + \texttt{daily\_trip\_count})$ given the feature matrix assembled above. The train–test split was performed on *time*: we sorted all unique dates chronologically, took the first 80% as training dates, and held out the remaining 20% as test dates. The test set therefore consists exclusively of future dates that the model never observed during training, directly mimicking the actual forecasting setting. A random split would leak future demand information into training through shared dates, producing optimistically biased test metrics. Within the training window, hyperparameter tuning used 3-fold cross-validation, preserving temporal order within each fold.

6.2 Models and Rationale

We trained four models spanning the bias–variance spectrum. **Ridge Regression** ($\alpha = 1$) serves as the linear baseline, establishing a lower bound on performance achievable without nonlinearity. **Random Forest** (300 trees, `max_depth=12`, `min_samples_leaf=5`) captures nonlinear relationships and interaction effects automatically; the depth constraint and leaf-size floor prevent overfitting to noisy high-demand observations. **Gradient Boosting** (300 estimators, learning rate 0.05, `max_depth=3`) builds shallow trees sequentially, correcting residuals from prior rounds; a low learning rate encourages generalization. **Tuned Random Forest** was selected via 3-fold `GridSearchCV` over `n_estimators` $\in \{100, 300\}$, `max_depth` $\in \{8, 12, \text{None}\}$, and `min_samples_leaf` $\in \{3, 5, 10\}$, scoring on negative RMSE; the best configuration was `n_estimators=300`, `max_depth=None`, `min_samples_leaf=3`.

7. MODEL EVALUATION AND SELECTION

7.1 Quantitative Performance

Table 2 reports MAE, RMSE, and R^2 on the held-out test set, computed on the original trip-count scale after inverting the log transformation.

Table 3: Top-10 feature importances from the Tuned Random Forest. Prior-day demand and weather dominate; spatial and cluster features provide complementary signal.

Feature	Importance
`lag_1_trip_count`	0.506
`mean_temperature`	0.194
`max_wind_speed`	0.075
`precipitation_sum`	0.061
`lag_7_trip_count`	0.056
`dow_sin`	0.020
`start_lat`	0.019
`start_lng`	0.018
`heavy_precipitation`	0.017
`station_cluster_0`	0.011

The Tuned Random Forest leads on all three metrics. Ridge Regression's negative R^2 (−0.151) means it performs worse than predicting the global mean: the log-transformed target contains interaction effects between lag demand, weather, and station identity that no linear model can capture without explicit cross terms. The gap between the two Random Forest variants (MAE 21.23 vs. 19.75) confirms that hyperparameter tuning provides meaningful gains within the same model family. Gradient Boosting's near-zero R^2 (0.038) suggests that the shallow-tree parameterization is not well-matched to a dataset where short-term autocorrelation creates sharp local structure best exploited by deeper partitions.

7.2 Feature Importance and Model Interpretability

Table 3 reports the top-10 feature importances from the Tuned Random Forest. The dominance of `lag_1_trip_count` (importance 0.506) confirms the strong day-to-day autocorrelation visible in the trend plot: prior-day demand alone accounts for over half the model's information gain. This has a direct operational interpretation, as the best single predictor of how busy a station will be tomorrow is how busy it was today. Mean temperature (0.194) ranks second, reflecting the tight coupling between cold-weather suppression and ridership that EDA revealed. Wind speed (0.075) and precipitation sum (0.061) follow, with lag-7 (0.056) close behind. Spatial features (`start_lat`, `start_lng`) together contribute approximately 0.037, encoding the geographic demand gradient across the service area. The cluster one-hot columns collectively add modest but non-trivial information beyond raw coordinates, validating the unsupervised pipeline's contribution to the supervised model.

7.3 Error Analysis by Demand Level

Table 4 decomposes test-set error by actual-demand quartile. The model is approximately unbiased for the lower three quartiles (MAE 14–17 trips). However, for the highest-demand quartile (mean actual ≈100

Table 4: Test-set performance decomposed by demand quartile. The model is well-calibrated for low to medium-high demand but systematically under-predicts the busiest station-days.

Demand Group	n	Avg Actual	Avg Pred.	MAE
Low	815	17.03	25.22	14.28
Medium-Low	855	41.17	46.98	16.64
Medium-High	801	61.48	60.65	15.98
High	788	99.90	83.14	32.62

trips/day), the model substantially under-predicts (mean predicted ≈83, MAE 32.6). This high-end bias is structurally expected: the log transformation compresses differences at the upper tail, tree models cannot extrapolate beyond the range of training splits, and extreme demand days are by definition rare in any 59-day training window. Operationally, predictions for Cluster 0 stations on optimal-weather weekdays should be treated as conservative estimates, and fleet managers should maintain buffer capacity at these locations.

7.4 Final Model Selection

We select the Tuned Random Forest as the final production model on three grounds. First, it achieves the lowest MAE and RMSE and the highest R^2 across the held-out future time period, demonstrating both accuracy and robustness to temporal drift. Second, unlike the linear baseline, it naturally captures the nonlinear weather–demand and lag–demand interactions that EDA identified as central to the problem; these interactions are not artifacts of overfitting but reflect genuine physical mechanisms. Third, the model provides interpretable feature importances that tell a coherent causal story, making its outputs actionable for non-technical operational stakeholders. The main limitation is the high-demand under-prediction bias documented above, which a quantile regression post-correction or a separate surge-prediction classifier could address in future work.

8. DISCUSSION

8.1 Key Findings

This project yields three substantive conclusions about Citi Bike demand. First, **demand exhibits strong short-term autocorrelation**: yesterday's trips at a station are by far the best single predictor of today's, accounting for over half the Random Forest's information gain. This implies that any operationally useful forecast system should prioritize real-time updating from the most recent observed demand rather than relying primarily on calendar or weather signals. Second, **weather effects are real but threshold-structured**: temperature and precipitation matter most through binary regime transitions (any precipitation vs. none; extreme cold vs. moderate cold) rather than through smooth linear relationships. Modeling them as continuous variables alone would capture only a fraction of their predictive value. Third, **station identity is an independent predictor**: even after controlling for location, weather, and calendar, the cluster label, which summarizes whether a station is a high-traffic commuter hub, a moderate commuter node, a residential mixed-use dock, or a park/leisure facility, provides additional explanatory power. This validates the pipeline design decision to bridge unsupervised clustering and supervised modeling.

8.2 Limitations

Several limitations constrain these conclusions. The two-month study window captures only a single winter season, meaning annual seasonality is entirely unobserved; a model trained on January–February data should not be expected to forecast May demand accurately without retraining. Holiday and special-event indicators were not included, likely explaining some high-residual days in the predicted-versus-actual scatter. The under-prediction of peak demand (MAE ≈ 33 trips for the top quartile) is a practical limitation for high-volume station management. Finally, the weather data represents a single city-wide daily average; station-level meteorological micro-variation is not captured.

8.3 Future Work

Several extensions could meaningfully improve performance. Extending the training window to a full year would allow the model to learn annual seasonality and holiday effects directly. A two-stage model, with a classifier identifying high-demand days followed by a quantile regressor calibrated for the top decile, would directly address the under-prediction bias. Incorporating real-time dock-level availability data or transit disruption indicators would capture supply-side constraints that currently appear as unexplained demand variability. Finally, deploying the model within an automated daily scoring pipeline would complete the transition from a research exercise to an operational tool.

9. INTERACTIVE DASHBOARD

To make the full data science workflow accessible beyond this report, we built a live Shiny for Python dashboard deployed at helenaliyz.shinyapps.io/citibike-dashboard. The dashboard presents five interactive tabs. The **Overview** tab displays key summary statistics and the system-wide daily demand trend with weekday/weekend coloring. The **EDA** tab offers on-demand chart selection: weekday/weekend comparison, weather scatter plots with trend lines, top-station rankings, and a correlation heatmap, all rendered with Plotly and configurable through sidebar controls. The **Clustering** tab shows the elbow and silhouette plots used to select $k = 4$, a PCA scatter of all 300 stations colored by cluster with

interactive hover labels, and the cluster summary table. The **Model** tab presents the four-model comparison table, feature importance bar chart, predicted versus actual scatter, and bias-by-demand-level visualization. The **Predictor** tab provides a live demand forecast: users specify a station cluster, approximate location, day of week, weather conditions, and prior-day demand, and the tuned Random Forest returns a predicted trip count with a contextual benchmark chart.

10. REPRODUCIBILITY

All code, data, and the report source are available at github.com/Helenaliyz/5243Project4. The pipeline comprises three sequential scripts: `data_acquisition_cleaning.ipynb` runs the web scraper, downloads zip files, cleans and aggregates trips, fetches weather, and saves the analytic CSV; `5243_project4_analysis.py` reads the clean CSV and produces all EDA figures and summary tables; `Project 4 Modeling.ipynb` performs clustering, feature engineering, model training, and evaluation. Python package requirements are listed in `requirements.txt`; the `README.md` provides step-by-step execution instructions and expected outputs.

11. TEAM CONTRIBUTIONS

Helena Li (yl6029) built the complete data acquisition and preprocessing pipeline, including the web scraper, Open-Meteo API integration, cleaning, aggregation, and the final analytic CSV. She developed and deployed the full interactive Shiny dashboard across all five tabs with a live demand predictor at shinyapps.io, and wrote and revised the full project report and repository README.
Wentao Zhong (wz2753) conducted the exploratory data analysis in Python, generating all EDA figures and summary statistics tables that underpin Section 3.
Kevin Ma (km4189) implemented the unsupervised clustering pipeline (K-Means, elbow/silhouette analysis, PCA visualization), carried out all feature engineering, trained and cross-validated all four supervised models, and produced the model evaluation tables and diagnostic plots.
Ketaki Dabade (kvd2112) contributed to the final written report, repository organization, interpretation of model evaluation results, and preparation of the oral presentation slides.